

Fundamentos de Java (Java Fundamentals)

Seção 7 Parte 1: Criando um Projeto de Inventário

Projeto

Visão Geral

Este projeto vai conduzi-lo ao longo das Seções 4, 5, 6 e 7 do curso. Após cada seção, outras etapas deverão ser concluídas até ser criado um aplicativo Java completo para manutenção de Inventário. Para cada parte, tome como base a parte anterior de modo que requisitos antigos e novos sejam atendidos. Inclua todas as partes em um pacote chamado Inventário.

Crie um programa de inventário que possa ser usado para uma ampla gama de produtos (cds, dvds, software etc.).

Tópico(s):

- Modificando Programas
 - Criando Métodos Estáticos (Seção 7.3)
 - Usando parâmetros em um método (Seção 7.1)
 - Retornar um valor de um método (Seção 7.1)
- Adicionando métodos (comportamentos) a uma classe existente (Seção 7.2)
- Implementando uma interface do usuário (Seções 5.1, 5.2, 6.2)

Instruções:

1. Abra o programa de inventário atualizado na **Seção 6: Criando um Projeto de inventário**.
2. Você vai modificar o seu código de modo que a classe principal não execute qualquer processamento, mas simplesmente chame métodos estáticos quando necessário.
 - a. Crie um método estático na classe **ProductTester** após o final do método principal chamado displayInventory. Esse método retornará valores e aceitará o array de produtos como um parâmetro. *Lembre-se:* quando especifica um array como um parâmetro, você utiliza o nome da classe como o tipo de dados, um par de colchetes vazios e o nome do array (`ClassName[] arrayName`)
 - b. Copie o código que exibe o array do método principal para o novo método displayInventory.
 - c. Onde você removeu o código de exibição do método principal, substitua-o por uma chamada ao método displayInventory. Lembre-se de incluir a lista de argumentos corretos correspondentes à lista de parâmetros no método que está sendo chamado.
 - d. Execute e teste o seu código
 - e. Crie um método estático na classe **ProductTester** após o final do método principal addToInventory. Esse método não retornará valores e aceitará o array de produtos e o scanner como parâmetros.
 - f. Copie o código que adiciona os valores ao array do método principal para o novo método addToInventory.
 - g. Para resolver os erros existentes no seu código, mova as variáveis locais necessárias (tempNumber, tempName, tempQty, tempPrice) do método principal para a parte superior do método addInventory.
 - h. Adicione uma chamada de método principal ao método addToInventory() de onde você removeu o loop for.

- i. Execute e teste o seu código
 - j. Crie um método em **ProductTester** para retornar o valor inteiro `getNumProducts()` que aceita o scanner como parâmetro. Mova o código inteiro que obtiver o número máximo de produtos do usuário para esse método, coloque uma chamada do método principal no seu novo método. Você armazenará o valor retornado na sua variável `maxSize`. Portanto, será necessário declarar 2 dessas variáveis: uma no seu método principal e outra no método `getNumProducts()`. Você pode remover o valor inicial de -1 da declaração no método principal.
 - k. Execute e teste o seu código
3. Crie dois novos métodos na classe **Produto**: um para permitir que o usuário adicione ao número de unidades em estoque (`addToInventory`) e outro para permitir que o usuário deduza do número unidades em estoque (`deductFromInventory`). Ambos os métodos deverão aceitar um parâmetro (quantidade) para armazenar o número de itens a serem adicionados/deduzidos. Coloque esses métodos sob os seus construtores.
4. Modifique a classe **ProductTester** para que o usuário possa ver, modificar ou descontinuar os produtos por meio de uma interface do usuário que é baseada em um sistema de menus.
 - a. Exiba um sistema de menus que mostrará opções e retornará a escolha de menu inserida pelo usuário.
 - i. O método terá o nome `getMenuOption`, retornará um valor inteiro e utilizará um objeto `Scanner` como parâmetro. Escreva o código sob o método principal.
 - ii. O menu deverá se parecer com:

```
1. Exibir Inventário
2. Adicionar Estoque
3. Deduzir Estoque
4. Descontinuar Produto
0. Sair
```

Insira uma opção de menu:
 - iii. Somente números entre 0 e 4 deverão ser aceitos. Qualquer outra entrada deverá forçar a exibição de um prompt ao usuário. **Lembre-se:** ao adicionar uma instrução de captura de tentativa (`try catch`), você deverá inicializar a variável para tudo o que falhar na condição `while`!
 - b. Crie um método que exibirá o valor de índice do array e o nome de cada produto, permitindo que o usuário selecione o produto que deseja atualizar (adicionar/deduzir).
 - i. O método terá o nome `getProductNumber`, retornará um valor inteiro e utilizará o array de produtos e um objeto `Scanner` como parâmetros. Ele deverá ter uma única variável local `productChoice` do tipo inteiro inicializada como -1. Escreva o código sob o método principal.
 - ii. Um loop `FOR` tradicional deverá ser usado para exibir o valor do índice e nome do produto. Use o tamanho do array para encerrar o loop. O nome de cada produto pode ser acessado por meio de seu método `getter` apropriado.
 - iii. O usuário só poderá inserir valores entre 0 e 1 menores que o tamanho do array. Todas as entradas deverão ter um tratamento apropriado de erros.

- c. Crie um método que adicionará valores de estoque a cada produto identificado.
- O método será denominado `addInventory`, não terá valor de retorno e utilizará o array de produtos e um objeto `Scanner` como parâmetros. Ele deverá ter duas variáveis locais `productChoice` que não precisam ser inicializadas e uma variável `updateValue` que deverá ser inicializada como -1. Ambas as variáveis locais deverão ter a capacidade de armazenar valores inteiros. Escreva o código sob o método principal.
 - Adicione uma chamada ao método `getProductNumber` especificando os parâmetros corretos e salvando o resultado na variável `productChoice`.
 - O usuário deverá receber a mensagem “Quantos produtos deseja adicionar?” e só poderá inserir valores positivos iguais ou maiores que 0. Todas as entradas deverão ter mensagens de erro apropriadas e um tratamento adequado de erros.
 - Assim que um valor de atualização válido tiver sido adicionado, os níveis de estoque do produto selecionado serão atualizados por meio do método `addToInventory` criado anteriormente. A variável `productChoice` é usada para identificar o valor do índice do produto no array e `updateValue` é a quantidade de estoque a ser adicionada.
- d. Crie um método que deduzirá valores de estoque para cada produto identificado.
- Siga o mesmo procedimento que o utilizado para adicionar estoque, mas atribua o nome `deductInventory` ao seu método. As restrições a essa entrada de dados são: o valor deverá ser igual ou maior que 0 e não poderá ser maior que a quantidade atual do estoque desse produto. Todas as entradas deverão ter mensagens de erro apropriadas e um tratamento adequado de erros. Use o método `deductFromInventory` para fazer a alteração no objeto `Produto` no array.
- e. A opção de menu final a ser implementada é a capacidade de marcar estoque como descontinuado.
- O método será denominado `discontinueInventory`, não terá valor de retorno e utilizará o array de produtos e um objeto `Scanner` como parâmetros. Ele deverá ter uma variável local com o nome `productChoice` que armazena um valor inteiro e que não precisa ser inicializada. Escreva o código sob o método principal.
 - Adicione uma chamada ao método `getProductNumber` especificando os parâmetros corretos e salvando o resultado na variável `productChoice`.
 - Agora utilize o método `setActive` para definir o valor de ativo como falso para o objeto `Produto` escolhido.
- f. Agora você precisa criar um método que junte tudo isso.
- O método deverá ter o nome `executeMenuChoice`, não terá valor de retorno e utilizará a opção de menu, o array de produtos e um objeto `Scanner` como parâmetros. Ele não requer variáveis locais. Escreva o código sob o método principal.
 - Utilize uma instrução de troca (`switch`) para executar os métodos criados nesse exercício. Para cada instrução `case`, utilize uma instrução de saída que exibirá um dos seguintes cabeçalhos antes de executar o método apropriado:


```
Exibir Lista de Produtos
Adicionar Estoque
Deduzir Estoque
Descontinuar Estoque
```

- g. O estágio final desse exercício é atualizar o método principal para fazer uso da nova funcionalidade. Atualize o seu código para que o seu método principal corresponda ao seguinte código:

```
public static void main(String[] args) {  
    //create a Scanner object for keyboard input  
    Scanner in = new Scanner(System.in);  
    int maxSize, menuChoice;  
  
    maxSize = getNumProducts(in);  
    if(maxSize ==0) {  
        //Display a no products message if zero is entered  
        System.out.println("No products required!");  
    }else {  
        Product[] products = new Product[maxSize];  
        addToInventory(products, in);  
        do {  
            menuChoice = getMenuOption(in);  
            executeMenuChoice(menuChoice, products, in);  
        }while(menuChoice!=0);  
    }//endif  
}  
//end method main
```

- h. Execute e teste o seu código.

5. Salve o seu projeto.